Université Lumière Lyon 2

Master SISE

Détection de Fruits par Deep Learning

UE Deep Learning for Computer Vision 2025/2026

Aya Mecheri

aya.mecheri@univ-lyon2.fr

Architecture :	YOLOv8n (Transfer Learning)
Tâche :	Détection d'objets
Dataset :	359 images, 5 classes de fruits
Date :	Février 2026

Table des matières

1 Introduction **2**

2 Description du Dataset **2**
2.1 Collecte des données 2
2.2 Classes et distribution 2
2.3 Annotation 2
2.4 Split train/validation/test 2

3 Architecture : YOLOv8 **3**
3.1 Présentation de YOLO 3
3.2 YOLOv8n 3
3.3 Transfer Learning 3

4 Hyperparamètres et Justifications **4**

5 Résultats Expérimentaux **4**
5.1 Résultats quantitatifs 4
5.2 Résultats qualitatifs 7
5.2.1 Cas réussis 7
5.2.2 Erreurs courantes 8

6 Discussion **9**
6.1 Analyse des résultats 9
6.2 Absence de surapprentissage 9
6.3 Influence du transfer learning 10
6.4 Limites et améliorations possibles 10

7 Conclusion **10**

Références **10**

1 Introduction

Ce rapport présente le projet réalisé dans le cadre de l'UE *Deep Learning for Computer Vision* (2025/2026). L'objectif est d'étudier une architecture de réseau neuronal profond de type CNN pour une tâche de **détection d'objets** sur un ensemble d'images personnalisé.

Nous avons choisi la tâche de **détection et classification de fruits**, en photographiant nous-mêmes 359 images contenant cinq types de fruits : pomme, banane, kiwi, citron et mandarine. Chaque image peut contenir plusieurs fruits simultanément, ce qui rend la tâche plus réaliste et difficile qu'une simple classification d'image.

L'architecture retenue est **YOLOv8n** (You Only Look Once, version 8), entraînée par *transfer learning* à partir de poids pré-entraînés sur le dataset COCO.

2 Description du Dataset

2.1 Collecte des données

Le dataset a été constitué entièrement par nos soins. Nous avons pris **359 photographies** avec un smartphone, dans différentes conditions d'éclairage et sur différents fonds, afin d'obtenir un dataset varié et représentatif. Chaque image peut contenir un ou plusieurs fruits de types différents.

2.2 Classes et distribution

Le dataset contient **5 classes** de fruits :

ID	Classe	Couleur	Caractéristiques
0	Pomme	Verte	Forme ronde, couleur variable
1	Banane	Jaune	Forme allongée, courbée
2	Kiwi	Marron	Forme ovale, texture rugueuse
3	Citron	Jaune vif	Forme ovale allongée
4	Mandarine	Orange	Forme ronde, couleur orange

Table 1 – Classes du dataset

2.3 Annotation

Les images ont été annotées manuellement avec l'outil **CVAT** (*Computer Vision Annotation Tool*), en dessinant des *bounding boxes* autour de chaque fruit. Les annotations ont été exportées au format **YOLO 1.1** (fichiers `.txt` avec coordonnées normalisées).

2.4 Split train/validation/test

Le dataset a été divisé aléatoirement en trois sous-ensembles :

Split	Nombre d'images	Proportion
Train	251	70%
Validation	53	15%
Test	55	15%
Total	**359**	**100%**

TABLE 2 – Répartition du dataset

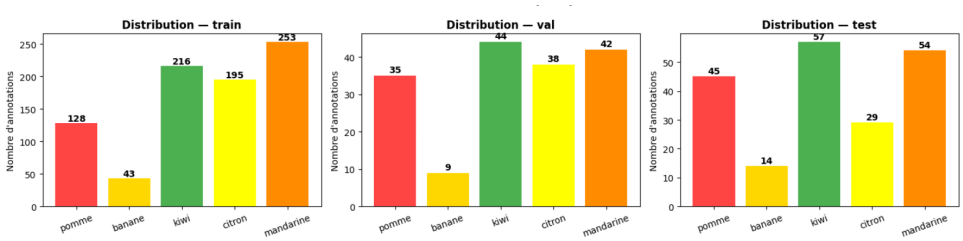

FIGURE 1 – Distribution des classes de fruits dans les trois splits du dataset

3 Architecture : YOLOv8

3.1 Présentation de YOLO

YOLO (*You Only Look Once*) est une famille d'architectures CNN dédiées à la détection d'objets en temps réel. Contrairement aux approches en deux étapes (comme Faster R-CNN), YOLO traite l'image en une seule passe (*single-stage detector*), ce qui lui confère une vitesse d'inférence élevée.

3.2 YOLOv8n

Nous utilisons **YOLOv8n** (variante *nano*), la version la plus légère de YOLOv8, développée par Ultralytics. Son architecture se compose de trois parties principales :

— **Backbone (CSPDarknet)** : extrait les caractéristiques visuelles de l'image à différentes échelles. Utilise des blocs C2f (Cross Stage Partial) pour un meilleur flux de gradient.

— **Neck (FPN + PAN)** : agrège les features maps de différentes résolutions pour détecter les objets de toutes tailles (petits, moyens, grands).

— **Head (Decoupled Head)** : prédit indépendamment les bounding boxes et les classes pour chaque grille de l'image.

3.3 Transfer Learning

Nous appliquons une stratégie de **transfer learning** (*fine-tuning*) : le modèle est initialisé avec des poids pré-entraînés sur **COCO** (80 classes, 330k images), puis adapté à notre tâche de détection de 5 fruits. Cette approche est particulièrement adaptée à notre dataset de taille modeste (359 images).

4 Hyperparamètres et Justifications

Hyperparamètre	Valeur	Justification
`epochs`	50	Suffisant pour le fine-tuning. L'early stopping empêche le surapprentissage.
`imgsz`	640	Taille standard YOLOv8. Bon compromis précision/vitesse.
`batch`	16	Adapté à la mémoire GPU T4 (16 GB).
`lr0`	0.01	Learning rate initial recommandé pour YOLOv8.
`optimizer`	AdamW	Meilleure convergence que SGD pour le fine-tuning sur petit dataset.
`patience`	15	Early stopping : arrêt si pas d'amélioration pendant 15 epochs consécutives.
`mosaic`	1.0	Combine 4 images en une. Utile car nos images contiennent plusieurs fruits.
`flipud/fliplr`	0.5	Flip horizontal/vertical. Augmente la diversité.
`degrees`	15.0	Rotation aléatoire $\pm 15°$. Invariance aux rotations.
`hsv_s/hsv_v`	0.7/0.4	Variation saturation/luminosité. Robustesse aux conditions d'éclairage.

Table 3 – Hyperparamètres d'entraînement et justifications

5 Résultats Expérimentaux

5.1 Résultats quantitatifs

Après fine-tuning, le modèle est évalué sur le **jeu de test** (55 images jamais vues pendant l'entraînement). La métrique principale est le **mAP** (*mean Average Precision*) :

— **mAP50** : mAP calculé avec un seuil IoU de 0.50 (standard PASCAL VOC)

— **mAP50-95** : moyenne du mAP sur les seuils IoU de 0.50 à 0.95 (standard COCO)

Métrique	Valeur (Test Set)
mAP50	**93.1%**
mAP50-95	**59.3%**
Precision	**91.1%**
Recall	**92.1%**

TABLE 4 – Résultats globaux sur le jeu de test

Résultats par classe

Classe	AP50	Barre	Commentaire
Pomme	**99.5%**		Excellente détection
Mandarine	**99.1%**		Excellente détection
Kiwi	**97.4%**		Très bonne détection
Citron	**95.6%**		Très bonne détection
Banane	**74.3%**		Plus difficile (forme allongée)
mAP50	**93.1%**		

TABLE 5 – AP50 par classe sur le jeu de test

Courbes d'entraînement et absence de surapprentissage

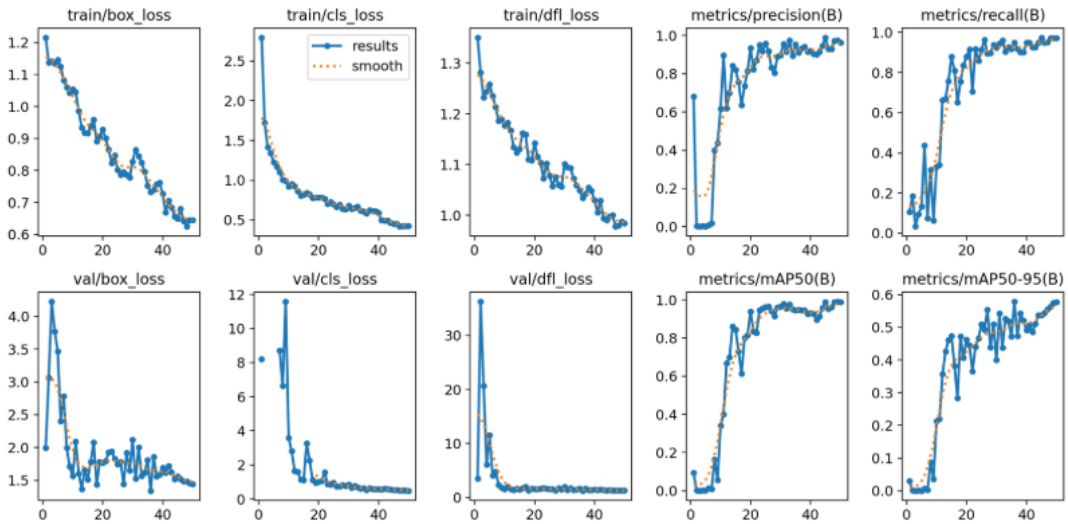

FIGURE 2 – Évolution des métriques pendant l'entraînement (50 epochs)

L'analyse des courbes d'entraînement (Figure 2) confirme l'absence de surapprentissage. Les *losses* d'entraînement et de validation descendent conjointement et de manière

cohérente tout au long des 50 epochs, sans divergence entre les deux courbes. Les métriques **mAP50** et **mAP50-95** progressent régulièrement sur la validation jusqu'à se stabiliser autour de 0.95 et 0.57 respectivement. Les pics observés durant les premières epochs (5–15) sont normaux et caractéristiques de la phase d'adaptation du transfer learning. La **Precision** et le **Recall** atteignent tous deux ∼0.95 en fin d'entraînement, confirmant un apprentissage sain et bien généralisé.

Matrice de confusion

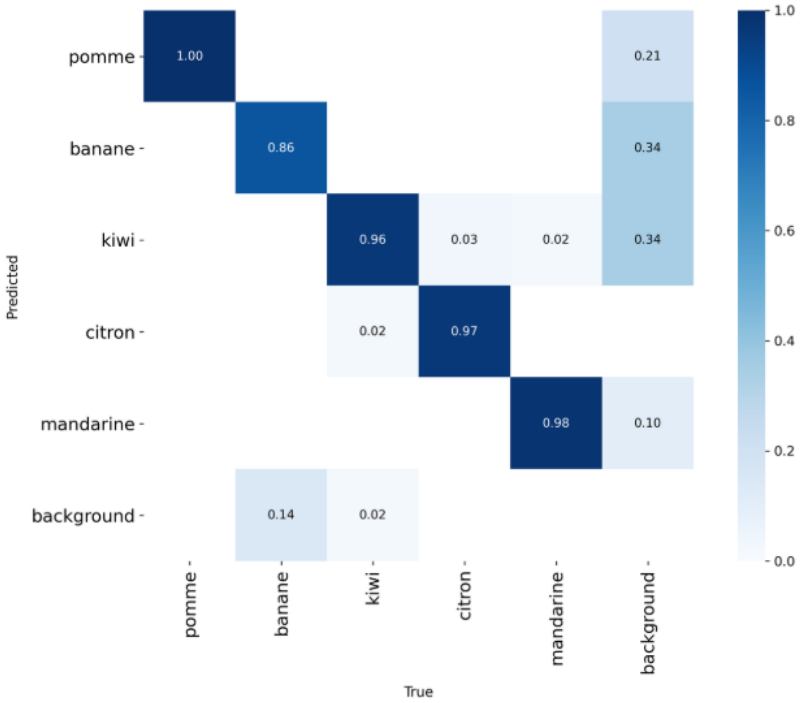

FIGURE 3 – Matrice de confusion normalisée — test set

Courbe Precision-Recall

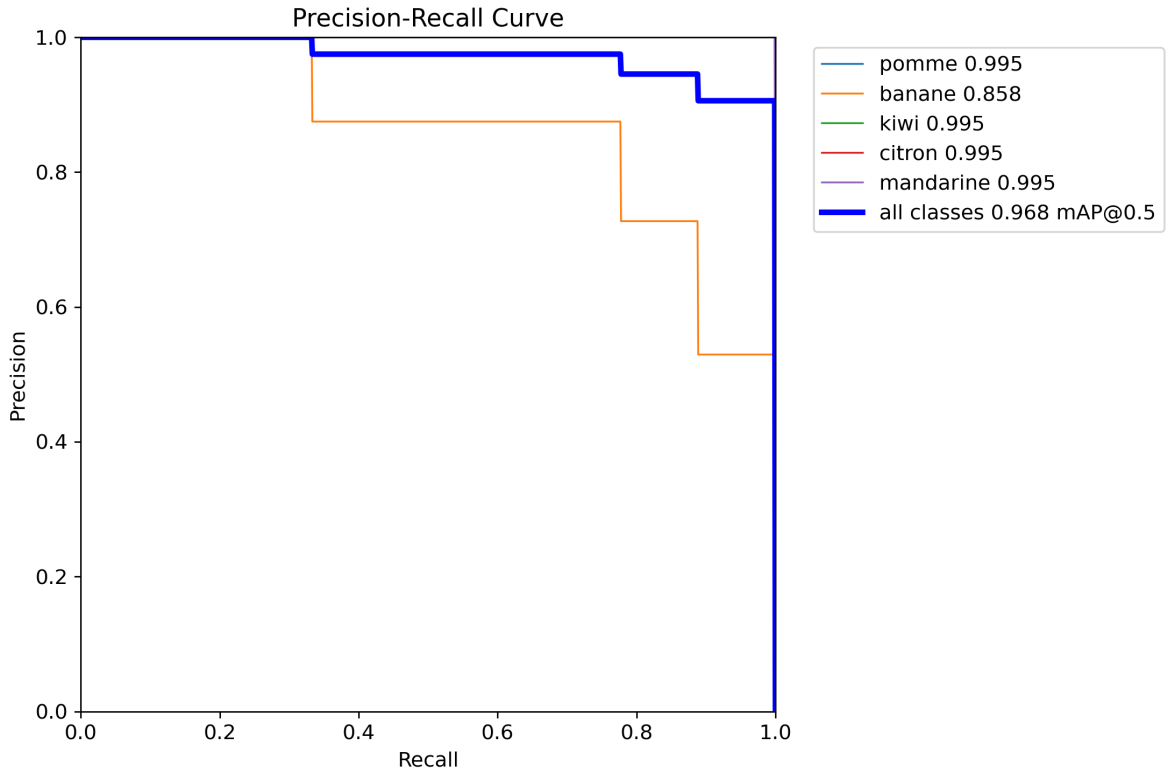

FIGURE 4 – Courbe Precision-Recall sur le jeu de test

5.2 Résultats qualitatifs

5.2.1 Cas réussis

Les figures suivantes montrent des exemples de détections réussies, où le modèle identifie correctement l'ensemble des fruits présents dans l'image avec une haute confiance.

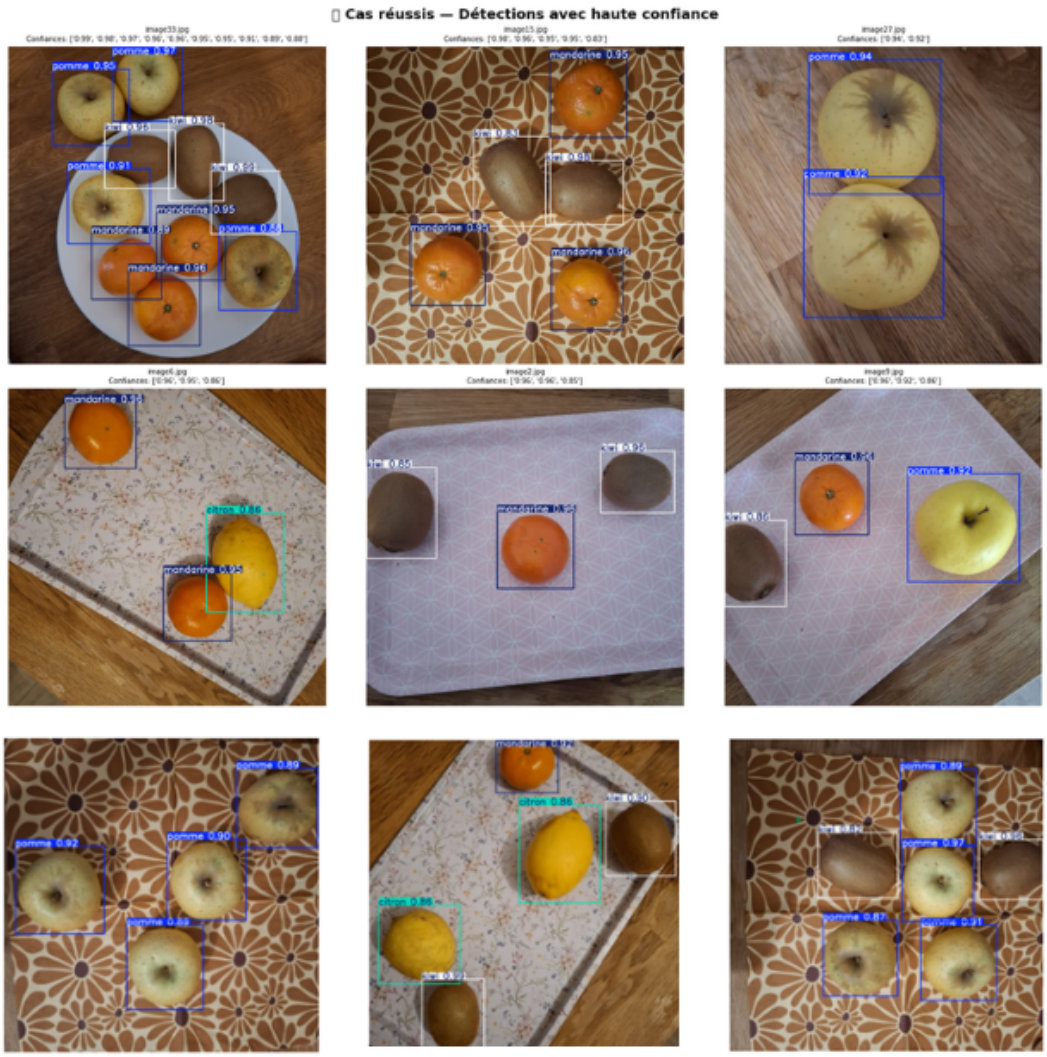

FIGURE 5 – Exemples de cas réussis — détections avec haute confiance

On observe que le modèle détecte correctement plusieurs fruits simultanément dans une même image, ce qui valide l'approche choisie. Les scores de confiance sont généralement supérieurs à 0.85 pour les détections correctes.

5.2.2 Erreurs courantes

Malgré les bons résultats globaux, le modèle commet certaines erreurs sur le jeu de test. L'analyse qualitative révèle trois types d'erreurs récurrentes :

— **Double détection (faux positifs)** : le modèle détecte parfois le même fruit deux fois avec deux bounding boxes superposées. Ce phénomène s'observe principalement sur les **bananes**, dont la forme allongée et courbée perturbe le mécanisme de suppression des doublons (NMS — Non-Maximum Suppression). Par exemple, une même banane est détectée avec des scores de 0.71 et 0.43 simultanément.

— **Détections avec faible confiance** : certaines détections présentent un score inférieur à 0.40, indiquant une incertitude élevée du modèle. Ces cas correspondent souvent à des fruits partiellement occultés ou placés en bordure d'image.

— **Chevauchement de bounding boxes** : sur les images contenant de nombreux fruits proches, les bounding boxes se chevauchent, rendant difficile la distinction entre objets individuels. Ce problème est particulièrement visible sur les images avec 6 fruits ou plus.

Ces erreurs sont caractéristiques des détecteurs mono-étape comme YOLO, et pourraient être réduites en ajustant le seuil NMS ou en augmentant le nombre d'images d'entraînement.

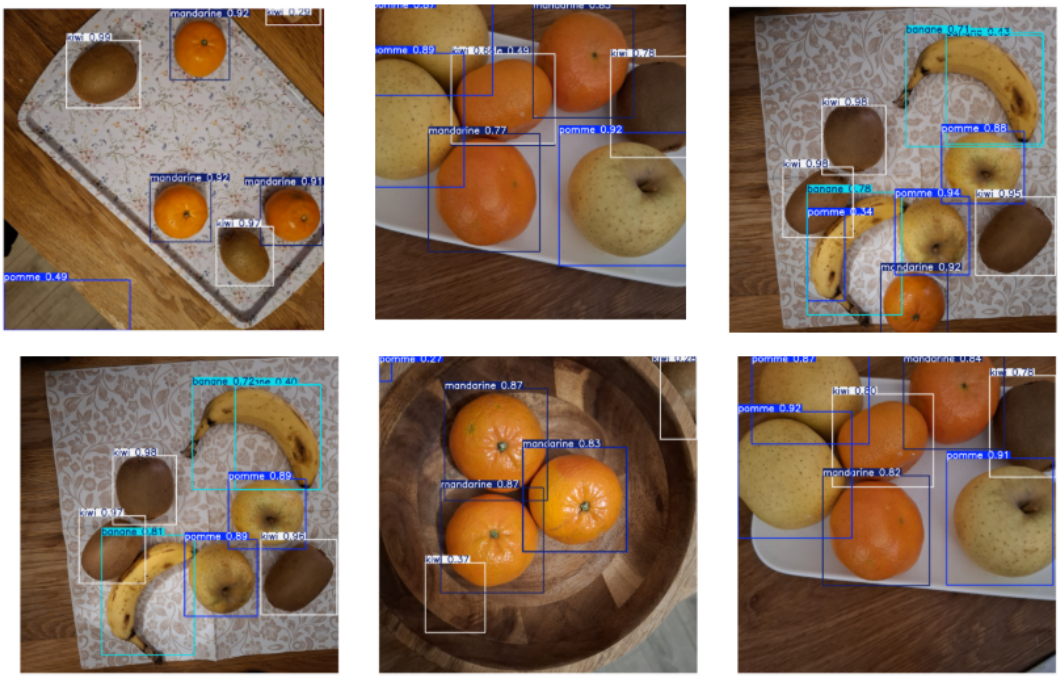

FIGURE 6 – Erreurs courantes — fausses détections

6 Discussion

6.1 Analyse des résultats

Les résultats obtenus sont satisfaisants compte tenu de la taille modeste du dataset. Le **mAP50 de 93.1%** sur le jeu de test témoigne de l'efficacité du transfer learning : partir de poids COCO pré-entraînés permet au modèle de converger rapidement et d'atteindre de bonnes performances malgré seulement 359 images d'entraînement.

6.2 Absence de surapprentissage

L'un des risques majeurs lors de l'entraînement sur un petit dataset est le **surapprentissage** (*overfitting*), où le modèle mémorise les données d'entraînement au lieu d'apprendre à généraliser. Dans notre cas, plusieurs éléments confirment que ce phénomène ne s'est pas produit :

— Les courbes de *loss* train et val **descendent ensemble** sans diverger — signe caractéristique d'un bon apprentissage.

— Le **mAP50 sur le jeu de test** (93.1%) est cohérent avec les performances sur le jeu de validation, confirmant que le modèle généralise bien sur des données jamais vues.

— Les techniques de **data augmentation** (mosaic, flip, rotation, variation HSV) ont joué un rôle clé en augmentant artificiellement la diversité du dataset, réduisant le risque de surapprentissage.

— Le mécanisme d'**early stopping** (patience = 15) aurait interrompu l'entraînement si les performances sur la validation avaient commencé à se dégrader.

6.3 Influence du transfer learning

Le fine-tuning depuis COCO s'est révélé particulièrement bénéfique car :

1. Le backbone pré-entraîné extrait déjà des features visuelles génériques (bords, textures, formes) sans avoir à les apprendre depuis zéro.
2. La quantité de données nécessaire est fortement réduite.
3. La convergence est plus rapide et plus stable.

6.4 Limites et améliorations possibles

— **Dataset limité** : 359 images reste modeste. Un dataset plus large améliorerait les performances, notamment pour la banane (AP50 = 74.3%).

— **Déséquilibre des classes** : si certaines classes sont sous-représentées, le modèle les apprend moins bien.

— **Modèle plus grand** : YOLOv8s ou YOLOv8m pourraient donner de meilleures performances au prix d'un temps d'entraînement plus long.

7 Conclusion

Ce projet a permis de mettre en pratique les concepts de *transfer learning* et de détection d'objets avec l'architecture YOLOv8. En partant d'un dataset de 359 images photographiées personnellement et annotées manuellement, nous avons obtenu un modèle performant avec un **mAP50 de 93.1%** sur le jeu de test, sans surapprentissage observé.

L'ensemble de la chaîne de traitement a été implémentée : collecte et annotation des données (CVAT), organisation du dataset (split train/val/test), fine-tuning du modèle (Google Colab, GPU T4), et évaluation quantitative et qualitative des résultats.

Références

1. Jocher, G. et al. (2023). *Ultralytics YOLOv8*. https://github.com/ultralytics/ultralytics
2. Lin, T.-Y. et al. (2014). *Microsoft COCO : Common Objects in Context*. ECCV 2014.
3. Redmon, J. et al. (2016). *You Only Look Once : Unified, Real-Time Object Detection*. CVPR 2016.

4. PyTorch. (2023). *TorchVision Object Detection Finetuning Tutorial*. https://pytorch.org/tutorials/
5. CVAT. (2023). *Computer Vision Annotation Tool*. https://app.cvat.ai